

code, circuits, & construction

code and fabrication resources for physical computing and networking

tigoe.net | [pcomp home](#) | [blog](#) | [code, circuits, & construction](#) | [my del.icio.us links](#)

HOME

PAGES

- About this site
- All About
- Microcontrollers
- Understanding Electricity
 - Breadboards
- Input & Output
 - Digital Input & Output
 - Analog Input
 - Analog Output
 - Controlling high-current circuits
- Serial Communication
 - Interpreting Serial Data
 - MIDI
- Motors
 - Controlling DC Motors

CATEGORIES

- arduino/wiring
- AVR
- BX-24
- circuits
- construction
- java
- Lantronix
- Lingo
- Max/MSP
- OSX
- pBasic (Basic stamp)
- Perl
- PHP
- PIC
- PicBasic Pro

Analog Output

Analog Output

Just as with input, there are times when we want an output with a range greater than 0 or 1. We might want to control the brightness of a lamp, for example, or the turn of a pointer on a dial, or the height of something hanging from a rope. In those cases, we need to make an analog output.

The most likely things that we might want to vary directly from a microcontroller are lights, sound devices, or things controlled by motors and other mechanical transducers (a mechanical transducer is something that converts electrical energy into mechanical energy — motors are the most common example). For the most part, there will be some other component in between the microcontroller and the final output device. There are lighting dimmers, motor controllers, and so forth, most of which can be controlled using some form of serial digital communication, which we'll talk about later.

The Arduino, the PIC, BX-24 and other digital microcontrollers can't produce a varying voltage, they can only produce a high voltage (in our case 5V) or low (in our case 0V). So instead, we "fake" an analog voltage by producing a series of voltage pulses at regular intervals, and varying the width of the pulses. This is called *pulse width modulation (PWM)*. The resulting average voltage is sometimes called a *pseudo-analog voltage*.

In the graph below, we pulse our pin high for the same length of time we pulse it low. The time the pin is high (called the pulsewidth) is about half the total time it takes to go from low to high to low again. This ratio is called the duty cycle. The duty cycle is

Processing
XBee

RSS FEEDS

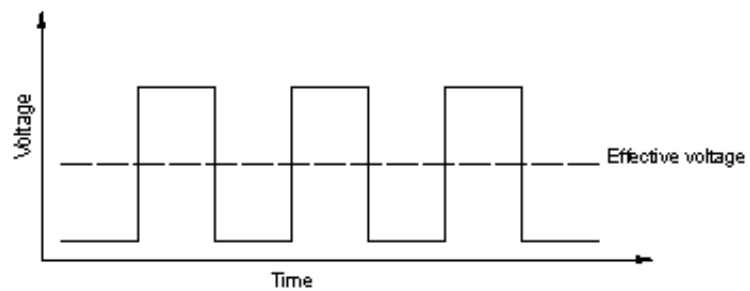
Posts
Comments

META

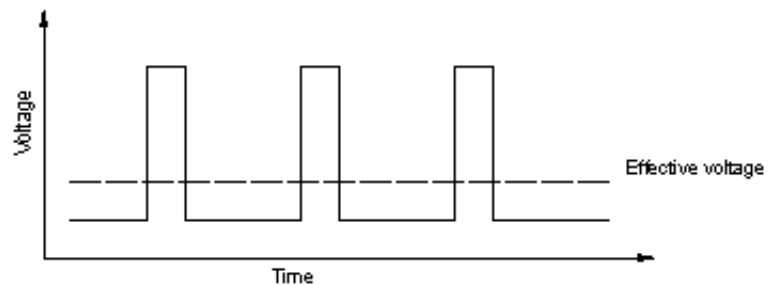
Log in

SEARCH

50%, and the average voltage is about half the total voltage.



If we make the duty cycle less than 50% by pulsing for a shorter amount of time than we pause, we get a lower effective voltage:



The pulsewidth is usually a very small time, on the order of a few microseconds or milliseconds at most.

On the PIC, the easiest way to create a modulated pulse is the `pulseout` command. It looks like this:

PicBasic Pro:

```
pulsout pin, pulsewidth
```

Pin refers to the pin you're going to pulse

Pulsewidth is the length of time each pulse takes, in seconds. It's dependent on your PIC's clock speed. For a 4MHz clock, the shortest possible pulse (e.g. period = 1) is 10 microseconds. With a 20MHz clock, it's 2 microseconds.

If the pin was high before the `pulsout` command, the pulse will go low and back to high. If the pin was low beforehand, the pulse will go high, then back to low.

On the BX-24, the pulse out command looks like this:

BX-24:

```
call pulseout(pin, pulsewidth, state)
```

Pin (a byte value) refers to the pin you're going to pulse

Pulsewidth (a single value) is the length of time each pulse takes, in seconds. The range is about 1.085 microseconds to 71.1 milliseconds (0.00002 to 0.071 seconds)

State (a byte value) is the state that the pin will be in when pulsed. If state = 1, the pin will normally be low, and pulse high. If state = 0, the pin will normally be high, and pulse low.

Wiring/Arduino:

On the Wiring and Arduino boards, there is no pulseout command. However, there are two ways to create a series of pulses: `analogWrite()` or writing your own pulseout.

analogWrite():

The pins marked PWM (pins 9 – 11 on Arduino, PWM 0 – 5 on Wiring) can be pulsed using the `analogOut()` command, like so:

```
analogWrite(pin, pulsewidth);
```

Pin refers to the pin you're going to pulse

Pulsewidth is a value from 0 – 255. 0 corresponds to 0 volts, and 255 corresponds to 5 volts. Every change of one point changes the pseudo-analog output voltage by 5/255, or 0.0196 volts.

Write your own pulseOut:

Here's an example of a `pulseOut()` function for Wiring/Arduino:

```
void setup() {
  // make pin 13 an output pin.
  // You'll pulse the LED on this pin:
  pinMode(13, OUTPUT);
}

void loop() {
  // read an analog input, 0 - 1023:
  int pulse = analogRead(0);
  // use that value to pulse an LED on pin 13:
  pulseOut(13, pulse, HIGH);
}

void pulseOut(int pinNumber, int pulseWidth, int state) {
  // only pulse if the pulseWidth value
  // is greater than 0:
  if (pulseWidth > 0) {
    // if the pulse should be high, go high then low:
    if (state == HIGH) {
```

```
        digitalWrite(pinNumber, HIGH);  
        delayMicroseconds(pulseWidth);  
        digitalWrite(pinNumber, LOW);  
        delayMicroseconds(pulseWidth);  
    }  
    // if the pulse should be low, go low then high:  
    else {  
        digitalWrite(pinNumber, LOW);  
        delayMicroseconds(pulseWidth);  
        digitalWrite(pinNumber, HIGH);  
        delayMicroseconds(pulseWidth);  
    }  
}  
}
```

For most applications, it's better to use the `analogWrite()` command, because the microcontroller maintains the PWM even after you give the command.

Applications of Analog Output

DC Motor Speed Control

Using a `pulseout` command, it is possible to vary the speed of a DC motor. We use the same transistor circuit as we did to turn on and off the motor, but instead of putting the output pin of the microcontroller high or low, we use `pulseout` command on it. The range of the required pulseout varies depending on the motor and the transistor used, but a range from 0.0002 to 0.02 is a good starting range. The varying pseudo-analog voltage on the base of the transistor creates a corresponding variance on the current flowing through the motor, and the motor spins at a variable rate.

For more on DC motor control, see the [DC motor control notes](#).

LED dimming

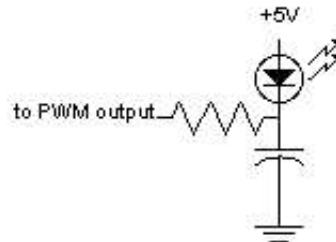
A `pulseout` command will also effectively dim an LED. However, an LED responds to the changing voltage much faster than a motor, and as the pulsewidth gets greater, you will notice the LED actually going on and off rather than dimming. A low pass filter circuit will smooth this somewhat.

Filter circuits are circuits which allow voltage changes of only a certain frequency range to pass. For example, a low-pass filter would block frequencies above a certain range. This means that if the voltage is changing more than a certain number of times per second, these changes would not make it past the filter, and only an average voltage would be seen.

Let's take our `pulseout` as an example. With a

pulsewidth of 1 millisecond, our voltage is going from high to low 1000 times a second. the frequency is 1000 changes a second, or 1000 *Hertz (Hz)*. If we had a filter circuit that blocked frequencies above 1000 Hz, we would see only an average voltage on the other side of the filter, instead of the pulses.

A basic low-pass filter consists of a resistor and a capacitor, connected like so:



The relationship between frequency blocked and the values of the capacitor and resistor used involve more complex math than we want to delve into here. If you're dimming an LED, start with a 10 μ f capacitor and a 220-ohm resistor, and experiment with different values from there to see what works best.

There is another command in PicBasic Pro (and pBasic and MBasic) that will pulse a pin multiple times, the `PWM` command. It's useful for producing smooth dimming, but it limits interactivity, because you can't do anything (like read sensors) in between pulses. Here's a [simple PWM example](#) in PicBasic Pro.

For more on filter circuits, see Paul Scherz' book, *Practical Electronics for Inventors*

Tone generation

Both PicBasic Pro and the BX-24 have a command, `freqout`, that allows you to generate a tone from any of the I/O pins. When a speaker or amplifier is connected properly to the pins, the tone can be heard.

in PicBasic Pro, the `freqout` syntax is as follows:

```
freqout Pin,duration ,freq1, freq2
```

The BX-24 syntax is as follows:

```
Call FreqOut(Pin, Freq1, Freq2, Duration)
```

Pin is a byte variable, the pin on which you want to generate the tone

Freq1 and *Freq2* are integer variables, the frequencies

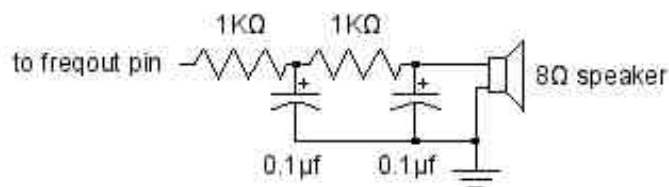
you want to generate. Units are in Hz. Human hearing extends from 20 – 20,000 Hz, so with an integer (whose values can be as high as 32,767), you can generate tones higher than the human ear can hear. Two tones can be generated at the same time; to generate a single tone, use the same tone for both frequency parameters.

Duration is the number of milliseconds to play the sound for. On the BX-24, it is a single precision floating point variable, the duration of the tone. It can range from 1 millisecond to about 2.56 seconds.

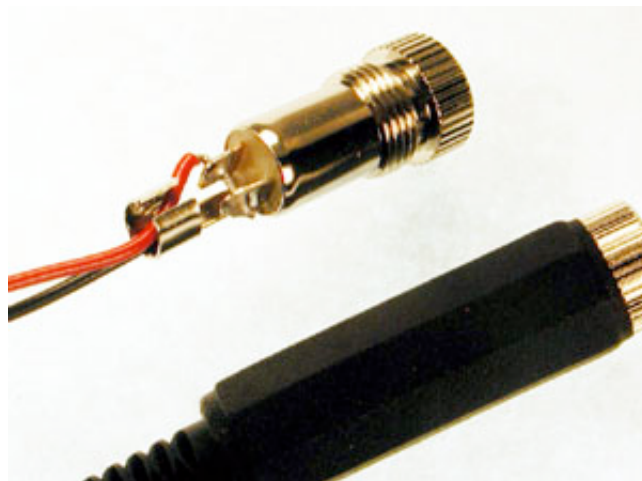
Wiring/Arduino:

There's no `freqout` command for Wiring/Arduino yet, but here's [an example of how to generate tones](#).

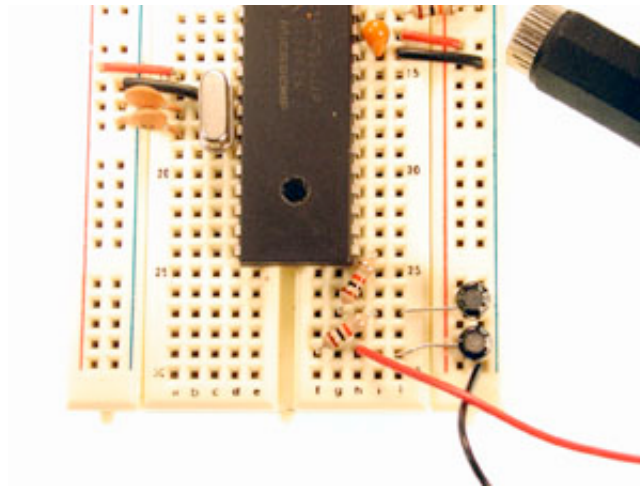
An 8-ohm speaker can be connected to the pin directly with a couple of capacitors, as shown:



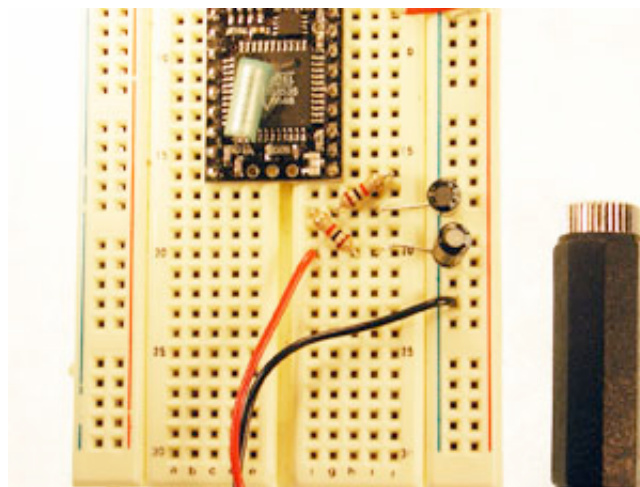
You can also replace the speaker in this circuit with an audio jack, and then connect it to an amplifier. For example, it can easily connect to most computer speakers with a female stereo mini-jack, as follows:



Stereo mini jack (female) and the connector sleeve. Note that the red lead is soldered to both the tip and ring solder tabs, and the black lead is soldered to the sleeve tab.



PIC and audio out circuit. Red and black leads go to stereo mini jack.



BX-24 and audio out circuit.

The tones from the microcontroller are not amplified, however, so you may find them to be difficult to hear. For better sound, wire your microcontroller to an audio jack so you can connect its output to the input of an amplifier.

For more on audio electronics, see Paul Scherz' book, *Practical Electronics for Inventors* (really, it's an excellent book!)

Servomotors

Perhaps the most exciting thing you can do as analog output is to control the movement of something. One simple way to do this is to use a servomotor.

Servomotors are motors with a combination of gears and an embedded potentiometer (variable resistor) that allows you to set their position fairly precisely within a 180-degree range. They're very common in

toys and other small mechanical devices. They're also very easy to control. They have three wires:

- power (usually +5V)
- ground
- control

Connect the +5V directly to a 5V power source that can supply at least one amp of current (don't try to power the motor from the microcontroller; it hasn't got enough power). Ground it to the same ground as the microcontroller. And attach the control pin to a pin on the microcontroller. Then you need to send a series of pulses to the control pin to set the angle. The longer the pulse, the greater the angle. To do this, call `pulseout`:

PicBasic Pro:

```
Pulsout pin, pulsewidth
```

BX-24:

```
call pulseout(pin, pulsewidth, state)
```

To pulse the servo, you generally give it a 5-volt, positive pulse between 1 and 2 milliseconds (ms) long, repeated about 50 times per second. The width of the pulse determines the position of the servo. Since servos' travel can vary, there isn't a definite correspondence between a given pulse width and a particular servo angle, but most servos will move to the center of their travel when receiving 1.5-ms pulses.

See the servo example for a detailed example for [Arduino/Wiring](#), the [PIC](#) or [BX-24](#).



(cc) tigoe | Thanks, [WordPress](#) | [Barthelme](#) theme by [Scott](#) | Valid [XHTML](#) & [CSS](#) | RSS: [Posts](#) & [Comments](#)